DECLARATIVE SYSTEMS · REFERENCE PACK

NIXOS CHEAT SHEET

Write down the state you want. Rebuild. The system makes a new generation. Every mistake has an escape hatch — that is the whole idea.

Beginner reference · unstable / 25.05 · verify options against your own release

DECLARATIVE FIRST | ROLLBACK ALWAYS | 27 SECTIONS | BEGINNER

00 • THE 30-SECOND VERSION

Command and option names drift between releases. When something differs on your machine, trust the official option search over any undated snippet.

```
sudoedit /etc/nixos/configuration.nix      # edit the system you want

environment.systemPackages = with pkgs; [  # add ordinary packages
  git
  firefox
  vlc
];

sudo nixos-rebuild test                    # try it for this boot only
sudo nixos-rebuild switch                  # keep it, and boot into it
sudo nixos-rebuild switch --rollback       # undo a bad generation
nix shell nixpkgs#ripgrep                  # borrow a tool, install nothing
```

• THE WHOLE IDEA

You never mutate the machine. You describe it, and rebuild. Every rebuild makes a new generation, and every old generation stays bootable until you deliberately delete it.

- search.nixos.org/packages — find the package attribute
- search.nixos.org/options — find the NixOS option

01 • THE MENTAL MODEL

Other distributions

Run an install command, and the current machine is mutated in place. State accumulates, and nobody quite knows what is on the box.

NixOS

Edit the configuration, build a generation, activate it. The description *is* the machine, and it is reviewable in Git.

- `/etc/nixos/configuration.nix` describes the system you want.
- `nixos-rebuild` evaluates that description and creates a generation.
- Packages live in the immutable `/nix/store`. Never edit it by hand.
- Adding a package does not overwrite another version of it.
- Uninstalling is deleting the line and rebuilding.
- Permanent changes belong in configuration, not in one-off commands.

Keep three scopes separate

System

> Software and settings that are part of *this machine*. `configuration.nix`, applied with `sudo nixos-rebuild switch`.

User

> One person's apps, shell, editor and dotfiles. **Home Manager** is the usual declarative tool.

Temporary

> Trying a command, or giving a project its compilers. `nix shell`, `nix run`, or a project dev shell.

02 • FILES AND PATHS THAT MATTER

PATH	WHAT IT IS
`/etc/nixos/configuration.nix`	Main system configuration.
`/etc/nixos/hardware-configuration.nix`	Generated hardware, filesystem and boot settings. Import it; do not casually rewrite it.
`/etc/nixos/flake.nix`	Optional flake entry point.
`/etc/nixos/flake.lock`	Exact input revisions. Commit it.
`/nix/store`	Immutable packages and build results. Never edit.
`/run/current-system`	The running generation.
`/nix/var/nix/profiles/system`	The current system profile.

Which style is this machine?

```
test -f /etc/nixos/flake.nix && echo "flake config present"
sudo nix-channel --list
```

• COMMON MISTAKE

Do not mix channel and flake update workflows. Find out which one your configuration actually uses, then follow only that path.

03 • THE MINIMUM NIX LANGUAGE

```
{ config, pkgs, ... }:

{
  networking.hostName = "my-computer";

  environment.systemPackages = with pkgs; [
    git
    curl
    firefox
  ];

  services.openssh.enable = true;
}
```

- Every assignment ends with a semicolon.
- Lists use brackets and whitespace – **no commas**.
- Attribute sets use braces; strings use double quotes.
- `#` starts a comment; a block comment is `/* … */`.
- `with pkgs; [ git curl ]` means `[ pkgs.git pkgs.curl ]`.
- Brackets and braces must balance.
- Package name is not always the command name: `ripgrep` gives you `rg`.

Format a file when `nixfmt` is available: `nixfmt configuration.nix`

04 • EDIT, TEST, APPLY

COMMAND	WHAT IT DOES
`nixos-rebuild build`	Builds only. Activates nothing.
`nixos-rebuild dry-activate`	Shows what activation would do.
`nixos-rebuild test`	Active now, but not the boot default.
`nixos-rebuild switch`	Active now *and* the boot default.
`nixos-rebuild boot`	Next boot only.

The beginner routine

1. Make one change you understand.
2. Run `sudo nixos-rebuild test`.
3. Confirm the application or service actually works.
4. Run `sudo nixos-rebuild switch`.
5. Commit the working configuration to Git.

• CONTEXT

For bootloader, filesystem, networking, display-manager or remote-SSH changes, keep local console access and a known-good generation. `test` lowers the risk; it cannot make every runtime change safe.

05 • INSTALLING AND REMOVING PACKAGES

System-wide and declarative – the normal way

```
nix search nixpkgs firefox
nix search nixpkgs ripgrep
```

```
environment.systemPackages = with pkgs; [
  git
  curl
  wget
  ripgrep
  firefox
  vlc
];
```

Apply with `sudo nixos-rebuild test`, then `switch`. To uninstall, remove the name from the list and switch. Files may linger in `/nix/store` until garbage collection, but the package is out of your system environment.

Module or plain package?

Check the **options** search before settling for a bare package. A `programs.*` module can add wrappers, permissions, services and integration that a package entry cannot.

```
programs.firefox.enable = true;
programs.steam.enable = true;
```

Per-user imperative profile – convenient, not declarative

```
nix profile install nixpkgs#ripgrep
nix profile list
nix profile upgrade '.*'
nix profile remove NAME_OR_INDEX
nix profile history
nix profile rollback
```

Do not mix `nix profile` with old `nix-env` in the same profile. For a reproducible machine, prefer configuration over accumulated profile installs.

Temporary

```
nix shell nixpkgs#ripgrep
nix shell nixpkgs#nodejs nixpkgs#git
exit

nix run nixpkgs#cowsay -- "hello from Nix"
```

• MEMORY HOOK

shell puts a tool *in your hands*; **run** uses it *once*; **systemPackages** makes it *part of the machine*. Try in a shell, then promote what you keep reaching for.

06 • GUI APPLICATIONS

```
environment.systemPackages = with pkgs; [
  firefox
  thunderbird
  vlc
  libreoffice
];
```

Rebuild, and the app appears in the desktop menu. If the menu looks stale, log out and back in; launch from a terminal to see real errors. Prefer a `programs.*` option when it provides integration you need — Steam is the classic example.

Unfree software

```
nixpkgs.config.allowUnfree = true;
```

A narrower `allowUnfreePredicate` is better when you only want specific packages allowed. Look the option up rather than copying an example — names matter.

Flatpak

```
services.flatpak.enable = true;

flatpak remote-add --if-not-exists flathub \
  https://flathub.org/repo/flathub.flatpakrepo
flatpak search SEARCH_TERM
flatpak install flathub APPLICATION_ID
flatpak run APPLICATION_ID
```

GNOME ships the portal integration; other desktops may need `xdg.portal` configuration. Flatpak apps are managed by Flatpak, not by `environment.systemPackages`.

AppImage

```
environment.systemPackages = with pkgs; [ appimage-run ];

appimage-run ./SomeApplication.AppImage
```

Random downloaded binaries

Prebuilt Linux binaries expect loaders and libraries at conventional FHS paths that NixOS does not provide. Work down this list in order:

1. The Nixpkgs package.
2. The application's NixOS module.
3. Flatpak, or `appimage-run`.
4. A Nix package or development shell of your own.
5. `programs.nix-ld.enable` – compatibility fallback only.

> • **TRAP**
>
> Do not train yourself to solve every problem with `curl | sudo sh`. Install scripts written for Ubuntu mutate paths that NixOS deliberately manages.

07 • SERVICES AND DAEMONS

```
services.openssh.enable = true;
sudo nixos-rebuild switch
```

```
systemctl status sshd.service
journalctl -u sshd.service -b
sudo systemctl restart sshd.service

systemctl --failed
systemctl list-units --type=service
systemctl list-timers
journalctl -b -p err
journalctl -f

systemctl --user status SERVICE_NAME
journalctl --user -u SERVICE_NAME
```

• COMMON MISTAKE

Never let `systemctl enable` be your source of truth. Enable the `services.*` option and rebuild; runtime start, stop and restart are for diagnosis only.

08 • NETWORKING AND FIREWALL

```
networking.networkmanager.enable = true;
networking.firewall.allowedTCPPorts = [ 22 80 443 ];
networking.firewall.allowedUDPPorts = [ ];
networking.firewall.enable = false;  # usually the wrong fix
```

```
ip address
ip route
resolvectl status
networkctl status
nmcli general status
ss -tulpn
```

Only the tools relevant to your configured network stack will apply.

09 • USERS, SHELLS, ENVIRONMENT

```
users.users.alice = {
  sc-camel-is-normal-user = true;
  description = "Alice";
  sc-camel-extra-groups = [ "networkmanager" "wheel" ];
};

programs.zsh.enable = true;
users.users.alice.shell = pkgs.zsh;

environment.variables = {
  EDITOR = "vim";
};
```

No passwords, tokens or private keys here – see section 18.

10 • HOME MANAGER: WHEN AND WHY

Home Manager declaratively manages one user's packages, dotfiles, shell, editor and user services. It is useful, and it is optional. Start with `configuration.nix`; add Home Manager when repeating manual dotfile work gets annoying.

```
home.packages = with pkgs; [
  ripgrep
  fd
];

programs.git.enable = true;
programs.zsh.enable = true;
```

```
home-manager build      # standalone
home-manager switch

sudo nixos-rebuild switch   # if integrated as a NixOS module
```

Keep Home Manager and Nixpkgs release branches matched, and read the official installation instructions before choosing standalone versus the module.

11 • CHANNELS VERSUS FLAKES

Both select the Nixpkgs revision used to evaluate your system.

Channels

Traditional. The channel state lives outside your configuration: simple, less explicit, less reproducible.

```
sudo nix-channel --list
sudo nixos-rebuild switch --upg

sudo nix-channel --update nixos
sudo nixos-rebuild switch
```

Channels are per-user: updating yours does not update root's.

Flakes

Inputs declared in `flake.nix`, revisions pinned in `flake.lock`. Excellent for reproducibility and Git.

```
nix.settings.experimental-featu
  [ "nix-command" "flakes" ];

sudo nixos-rebuild switch \
  --flake /etc/nixos#HOSTNAME
sudo nixos-rebuild switch --fla
```

Some commands still label this interface experimental.

```
nix flake update            # every input
nix flake update nixpkgs    # just nixpkgs
nix flake show
nix flake check
nix flake metadata

git diff flake.lock
sudo nixos-rebuild test --flake .#HOSTNAME
sudo nixos-rebuild switch --flake .#HOSTNAME
```

• THE GIT TRAP

A flake inside a Git repository ignores files Git does not know about. You need not commit before building, but the file must be tracked: `git add path/to/new-module.nix`.

12 • UPDATING NIXOS

```
# channels
sudo nixos-rebuild switch --upgrade

# flakes
cd /path/to/config-repository
nix flake update
sudo nixos-rebuild test --flake .#HOSTNAME
sudo nixos-rebuild switch --flake .#HOSTNAME

nixos-version
```

Updating packages within a release is not the same as changing release branches. Before a release change:

- Read both releases' notes.
- Change the channel URL or flake input branch deliberately.
- Update matching Home Manager inputs.
- Build and test before removing old generations.
- Keep recovery media for machines you care about.

13 • STATEVERSION

`system.stateVersion` and `home.stateVersion` select compatibility defaults for stateful data and module behaviour. They do **not** pin your installed version.

- [ ] Keep the value generated during the original installation
- [ ] Keep the Home Manager value chosen at initialisation
- [ ] Do not bump either because you upgraded
- [ ] Change one only after reading its documentation and migration notes

14 • ROLLBACKS AND GENERATIONS

```
sudo nixos-rebuild switch --rollback
sudo nixos-rebuild list-generations
```

If the machine will not boot

1. Reboot.
2. Open the bootloader's NixOS generations submenu.
3. Select a known-good older generation.
4. Fix `configuration.nix`, or restore the old `flake.lock`.
5. Rebuild a corrected generation.

After booting an older generation, make it the next boot default if you need to:

```
sudo /run/current-system/bin/switch-to-configuration boot
```

• KEY IDEA

Rollback changes only the *active* generation. Fix the source configuration too, or the next rebuild recreates the same failure.

15 • FINDING NAMES AND DOCUMENTATION

```
nix search nixpkgs SEARCH_TERM
man configuration.nix
nixos-help
man nixos-rebuild
nix --help
nix SUBCOMMAND --help

rg 'firefox|flatpak|pipewire' /etc/nixos
command -v firefox
readlink -f "$(command -v firefox)"
```

• CONTEXT

Confirm any online example targets your release. Wiki pages, forum posts and old configuration repositories are full of renamed and removed options. The option search and the manual for *your* release are stronger evidence than an undated snippet.

16 • SPLITTING A GROWING CONFIGURATION

```
imports = [
  ./hardware-configuration.nix
  ./packages.nix
  ./desktop.nix
  ./services.nix
];
```

```
# packages.nix
{ pkgs, ... }:

{
  environment.systemPackages = with pkgs; [
    git
    curl
    ripgrep
  ];
}
```

NixOS merges compatible option definitions from imported modules. Splitting by topic makes changes easy to review and easy to undo.

17 • PROJECT DEVELOPMENT ENVIRONMENTS

```
nix shell nixpkgs#nodejs nixpkgs#jdk nixpkgs#git
nix develop      # flake devShell
nix-shell        # traditional shell.nix

programs.direnv.enable = true;
programs.direnv.nix-direnv.enable = true;  # Home Manager
direnv allow
```

Do not install every compiler and SDK globally because one project needs it.

• TRAP

Never approve an unfamiliar `.envrc`, `flake.nix` or `shell.nix`. It is executable code that runs as your user.

18 • SECRETS: KEEP THEM OUT OF NIX

• DO NOT

The Nix store is not a secret store – its paths are readable by local users. Never write plaintext passwords, API tokens, private keys or environment-file contents into `configuration.nix`, `home.nix`, a flake input, `environment.variables`, or a string generated into a service file. Do not assume `builtins.readFile` makes a secret safe – evaluation can copy the result into store artifacts.

Beginner-safe approaches

Provision it outside the store

> Keep the secret on disk with strict Unix ownership and permissions, and point the service's `*File` option at it.

Declarative and encrypted

> Learn **sops-nix** or **agenix**: encrypted material lives in the repository and is decrypted during activation into a restricted location such as `/run/secrets`.

Prefer File options

> Options ending in `File`, `passwordFile`, `tokenFile`, `environmentFile` or `credentialsFile`. Names are not universal – search the option documentation for your service.

19 • GARBAGE COLLECTION AND DISK SPACE

```
df -h
du -sh /nix/store
nix path-info -Sh /run/current-system

nix-collect-garbage
sudo nix-collect-garbage
sudo nix-collect-garbage --delete-older-than 30d
sudo nix-collect-garbage -d
```

```
nix.gc = {
  automatic = true;
  dates = "weekly";
  options = "--delete-older-than 30d";
};

nix.settings.auto-optimise-store = true;
sudo nix store optimise
```

• -D REMOVES YOUR ESCAPE HATCH

`-d` deletes rollback history. Confirm the current system works before using it, especially after kernel, bootloader, filesystem or release changes.

20 • WHEN A REBUILD FAILS

```
sudo nixos-rebuild switch -L --show-trace
```

Usual suspects

- Missing semicolon.
- Unbalanced `]`, `}` or `)`.
- Commas used inside a Nix list.
- Misspelled package attribute or option name.
- Option renamed between releases.
- Package is unfree and `allowUnfree` is not configured.
- Flake references a newly created file that Git does not track.
- Wrong hostname after `#` in `--flake .#HOSTNAME`.
- Home Manager and Nixpkgs release inputs do not match.
- A copied module assumes `x86_64-linux` on an `aarch64-linux` machine.

```
nixos-version
hostname
git status
nix flake show
systemctl --failed
journalctl -b -p err

sudo nixos-rebuild switch --repair   # only for a damaged store
```

• SHORTCUT

Read the *first* meaningful error in the output. Everything after it is usually noise, and `--repair` is almost never the answer to a syntax or option error.

21 • THE PROGRAM IS NOT FOUND

```
command -v PROGRAM_NAME
echo "$PATH"
hash -r
```

- Package name and executable name differ.
- The rebuild failed, so the new generation never activated.
- It went into a different user's profile.
- Your shell cached a failed lookup – run `hash -r`.
- A GUI menu needs a logout to refresh.
- The package provides a library, plugin or service rather than a command.

22 • KEEP THE CONFIGURATION IN GIT

```
git status
git diff
git add configuration.nix flake.nix flake.lock
git commit -m "describe the working system change"
```

EDIT TEST SWITCH VERIFY COMMIT

Restoring a previously committed `flake.lock` restores your input pins. A private repository is still not a reason to commit plaintext secrets.

23 • THINGS NOT TO DO

- Do not manually modify `/nix/store`.
- Do not expect apt, dnf, pacman or Homebrew tutorials to map onto NixOS.
- Do not use `sudo nix profile install` for ordinary system software.
- Do not use `systemctl enable` as the permanent source of truth.
- Do not paste unknown overlays or `allowUnsupportedSystem` hacks to make an error disappear.
- Do not run garbage collection with `-d` immediately after a risky upgrade.
- Do not change `system.stateVersion` to match the newest release.
- Do not store plaintext secrets in Nix configuration.
- Do not regenerate `hardware-configuration.nix` casually.
- Do not run a project's flake, `shell.nix` or `.envrc` without treating it as executable code.

24 • QUICK DECISION GUIDE

WHAT YOU WANT	WHAT TO REACH FOR
This package permanently, for the whole machine	`environment.systemPackages` or a `programs.*` option, then `switch`
This GUI application	Same as any package — check for a `programs.*` module first; Flatpak is a fine alternative
To try a command without installing it	`nix shell nixpkgs#PACKAGE`
To run one packaged command once	`nix run nixpkgs#PACKAGE`
Only my user, with declarative dotfiles	Home Manager
Tools only while working on this project	`nix develop`, a devShell, or `nix-shell`
I changed configuration.nix — what now?	`nixos-rebuild test`, then `switch`
I enabled a service	Rebuild, then `systemctl status` and `journalctl`
The new generation is broken	`switch --rollback`, then fix the source
The machine will not boot	Select an older generation in the bootloader
Package updates	Channels: `switch --upgrade` · Flakes: `nix flake update`, then rebuild
The disk is filling up	Inspect, verify the current generation, then collect garbage

25 • COMMAND MINI-CARD

Rebuild

```
sudoedit /etc/nixos/configurati
sudo nixos-rebuild build
sudo nixos-rebuild dry-activate
sudo nixos-rebuild test
sudo nixos-rebuild switch
sudo nixos-rebuild boot
sudo nixos-rebuild switch --rol
sudo nixos-rebuild list-generat
```

Packages and flakes

```
nix search nixpkgs SEARCH_TERM
nix shell nixpkgs#PACKAGE_NAME
nix run nixpkgs#PACKAGE_NAME
nix profile list

nix flake show
nix flake check
nix flake update
sudo nixos-rebuild switch --fla
```

Diagnose

```
systemctl --failed
systemctl status SERVICE_NAME
journalctl -u SERVICE_NAME -b
journalctl -b -p err
```

Inspect and clean

```
nixos-version
command -v PROGRAM_NAME
readlink -f "$(command -v PROGR
nix-collect-garbage
sudo nix-collect-garbage --dele
```

26 • OFFICIAL REFERENCES

- NixOS manual – nixos.org/manual/nixos/stable/
- Nix reference manual – nix.dev/manual/nix/latest/
- First steps – nix.dev/tutorials/first-steps/
- Package search – search.nixos.org/packages
- Option search – search.nixos.org/options
- Official wiki – wiki.nixos.org

- Home Manager manual — nix-community.github.io/home-manager/
- Secret managers compared — wiki.nixos.org/wiki/Comparison_of_secret_managing_schemes

> *If you want a lasting change, describe it in configuration and rebuild. If you only need a tool briefly, use* `nix shell` *or* `nix run`*. If a change fails, your previous generation is still your escape hatch — until you deliberately garbage-collect it.*
>
> FINAL RULE OF THUMB

Print and tick: my next change

- [ ] One change, written down and understood
- [ ] `nixos-rebuild test` passed
- [ ] The thing I wanted actually works
- [ ] `nixos-rebuild switch`
- [ ] Committed to Git

Every callout colour means the same thing on every page: magenta is the thing to remember, blue is context, cyan is a trick, coral is a trap, periwinkle is a worked example, violet is a self-test.